

所属类别	2024 年“华数杯”全国大学生数学建模竞赛	参赛编号
本科组		CM2402873

基于多目标规划的景点游览线路

摘要

在互联网信息全球化与数字化的背景下，随着“city 不 city”这一网络流行语的热度攀升，越来越多的外国游客在中国境内进行旅行并在网络平台上进行分享，展示了丰富多彩的中国文化和风景，推动了旅游业的发展。随着中国过境免签政策的实施，外国游客的流入量显著增加，为此，如何优化游客的旅行体验，成为一个值得研究的问题。本研究针对 352 个城市的 35200 个旅游景点数据集，根据外国游客的要求，进行最优优化路线设计。

考虑到外国游客在中国境内可逗留 144 小时，且选择任一城市的机场出境，提出了“城市最佳景点游览原则”，即每个城市仅选择评分最高的景点进行游玩。现有覆盖 352 个城市的数据集，包含每个城市 100 个景点的信息，包括评分、开放时间、门票等。

针对问题一：首先，在分析所有景点评分过程中，发现存在缺少评分以及评分不标准的景点，对此团队通过附件中的网址搜索对其进行了补充。记录每个城市获得该最高评分的景点数量，再将其累加，最后按数量排序列出前 10 个城市。

针对问题二：以**熵权法**为主要解题思路，本题中涉及综合评价 352 个城市，需要考虑城市规模等因素，仅靠常识很难告诉人们各指标对城市游览的重要程度如何衡量、如何评价各城市。因此，团队采用熵权法，对每个因素进行定量统计分析。选出综合得分最高的前 50 个城市，作为外国游客最向往的城市列表。

针对问题三：以**混合整数线性规划（MILP）**为主要解题思路，统计相关数据。为确保游客在 144 小时以内游玩尽可能多的城市，因此将是否游览某一景点作为决策变量，花费时间及费用为连续变量。确定目标为将游览城市数量最大化、游玩体验最优化，创建一个基于整数线性规划的旅行问题模型，并同时考虑到旅行时间和门票费用的约束。

针对问题四：问题四仍然沿用问题三的解题思路，以**MILP**为主要解题思路，使用前面问题统计出的相关数据，目标确定为最大化游览城市数量的同时最小化总费用（即门票和交通费用）。因此在构建模型时，决策变量仍为是否游览某个城市，用 0 或 1 表示，使用合适方法进行求解即可。

针对问题五中，以**多目标规划遗传算法 NSGA-II**为主要解题思路，问题五具有多个冲突目标，需要同时考虑旅行时间的 144 小时限制、最小化旅行成本和最大化访问的城市数量，因此我们需要多目标规划模型。此外，由于本题的出发城市随机，因此需要动态调整路线以达到最优化目的。由于遗传算法具有较强的全局搜索能力，能够在解决不同类型的问题时进行调整和优化，因此采用多目标规划遗传算法 NSGA-II 模型，得到最优路径。

关键词：熵权法 混合整数线性规划（MILP） 多目标规划遗传算法 非支配排序遗传算法 NSGA-II

一、问题重述

1.1 问题背景

随着数字化时代的发展，社交媒体对信息的传播与接受正在发生显著变化。尤其是伴随着“city 不 city”这一网络流行语的爆火，越来越多人的关注。随着过境免签政策的实施，越来越多外国游客选择来华旅游，通过各种网络平台分享他们在中国的旅行体验，展现了一个真实而丰富的中国形象，使得这一趋势具有多方面的积极效果。

在游客的旅行过程中，他们可能面临众多城市和景点的选择问题。根据游客的入境限制，这些外国游客在中国境内可逗留 144 小时，并可从任一城市的机场出境。中国境内有 352 个城市，每个城市的旅游景点数量众多，为了提高游客的游览效率，本题假定“城市最佳景点游览原则”，即每个城市只选择最高评分的一个景点游玩。

目前，我们拥有一个涵盖 352 个城市的旅游景点数据集，每个城市中包含 100 个景点的详细信息，包括名称、评分、开放时间等。通过有效运用这些数据，分析并解决一系列复杂的旅游规划问题，将有助于提升外国游客的旅行体验。同时，通过对城市综合评分的评估，可以更好地帮助游客选择最具吸引力的城市游览，合理安排时间和预算，从而实现高效的旅游体验。

面对考虑到各类因素的旅游需求，如城市规模、交通便利性以及环境影响等，我们亟需通过数学建模与优化算法，针对这 352 个城市的景点进行系统研究。目标是为外国游客规划出最佳的旅游路线，满足他们在短时间内尽可能游览多个城市的需求，并在确保游览体验的基础上，降低总费用和出行时间。在这一背景下，研究不仅关乎游客的个体体验，更将对中国旅游业迎接全球化挑战、提升国际形象、促进文化交流具有重要的现实和理论意义。

1.2 问题提出

问题一：该问题要求分析 352 个城市中 35200 个景点评分的数据以得出的一系列统计结果。首先找出所有景点评分中的最高分。在全国范围内，统计获得最高评分的景点数量。确定哪些城市中获评最高评分的景点数量最多。根据拥有获得最高评分的景点数量对城市进行排序，并列出前十个城市及其对应的数量。

问题二：题目设定外国游客遵循“城市最佳景点游览得原则”即“每个城市只选择一个评分最高的景点游玩”，需要结合城市规模、环境环保、人文底蕴、交通便利、以及气候、美食等因素，对 352 个城市进行综合评价，并选出“最令外国人向往的 50 个城市”。此处，以下几个疑问有待解决：（1）如何处理这几个因素？（2）评价的方法如何选择？（3）排序的依据是什么？（4）每个城市的最佳游览景点是随机选择还是有其它要求？

问题三：题目的前提要求：在问题二得到的 50 个城市的基础上，只用高铁的交通方式进行游览，因此在规划 144 小时以内使游玩城市数量最大化的路线时，应该思考以下问题：（1）如何处理没有高铁的城市？（2）如何计算总费用？（3）如何定义此模型？

问题四：题目的要求是在问题三的情景下，游览目标变为了游览尽可能多的城市，同时要求综合游玩体验最好，在规划出合适路线后，给出该路线中门票和交通的总费用，总花费时间以及可以游玩的城市数量，需要思考的问题有：（1）游览尽可能多的城市

和花费尽可能少的费用在求解时是否会产生冲突？（2）目标函数需要进行哪些修改？（3）约束条件需要进行哪些修改？

问题五：一名外国游客计划在中国游览山景，入境后有 144 小时的时间。在每个游览的城市中选择评分最高的一座山景点，同时要求通过高铁进行城市之间的交通，以最小化门票和交通总费用，尽可能多地游览山景。定制一个 144 小时的旅游路线。路线需要包括每个城市的评分最高的山景点，高铁交通方案，总花费时间，门票和交通的总费用，以及可以游览的景点数量。

二、问题分析

2.1 问题一分析：

问题一要求对附件数据进行一系列统计，以得到与最高评分景点数量的城市排列。首先需要从指定目录中读取所有 CSV 文件，对其进行数据处理。其中针对评分列存在缺失值，需要进行搜索补充，若仍存在无效数据，则进行了转换处理。针对名字列，需要避免重复数据的出现。其次，在遍历每个城市的评分数据时，记录当前的最高评分，并在每个文件的基础上筛选出所有获得该评分的景点。然后对每个城市的最高评分景点数量进行计数，并累计全国范围内获得最高评分的景点数量。最后将所有城市以获得最高评分景点的数量进行降序排序，并输出前十的城市及其景点数量。

2.2 问题二分析：

问题二要求结合城市规模、环境环保、人文底蕴、交通便利、以及气候、美食等因素，对附件中 352 个城市进行综合评价，并选出“最令外国人向往的 50 个城市”。在建立模型前，本团队先对各因素对游览城市的影响进行了量化。由于人口多的城市通常拥有更大的旅游市场基数对应着更大的城市规模，能吸引更多的游客，因此，将各个城市的人口数量来量化城市规模，同理，将绿化占比作为量化环境环保的依据，将每个城市所有景点的总评分作为人文底蕴的初始数据，用各城市高铁线路数量量化交通便利度。除此，各地区由气象灾害而导致的经济损失侧面反映了气候的恶劣程度，以及各城市餐饮业的增速反映了美食的发展程度。

而后，团队对各因素数据进行处理，考虑到若仅靠主观想法就直接衡量各因素对城市游览的重要程度时，结果存在极大的不准确性，因此，团队首先对于每个指标列（如高铁线路数量、气候受灾损失、人文底蕴等），计算其数据的概率分布，并计算其信息熵，将熵值作为多样性的度量。然后，根据熵值与权重之间的关系公式，计算出每个指标的权重。再进行归一化处理，之后进行综合评分排序。最后，随机生成这 50 个城市对应的评分最高的景点。

2.3 问题三分析：

问题三要求规划游玩路线，首先，需要获得问题二的最令外国游客向往的 5050 个城市的各种时间、费用数据，对于门票费用和高铁费用，因为每个城市只选择一个城市游玩，所以无需考虑城市内的交通费用，因此，交通费用支出仅来自于高铁票钱。其次，引进二进制变量 0、1 表示某城市游玩与否，确定目标函数即最大化游览的景点数量以及约束条件。

2.4 问题四分析：

问题四与问题三类似，仍要求规划游玩路线，只是改变了目标函数和约束条件。首先所需要的数据应该与问题三中一致，即最令外国游客向往的 50 个城市各个时间、费用数据，对于门票费用和高铁费用，随后仍然使用问题三中的方法，不改变决策变量，用相同的方式求解新的目标函数下的最优解，即为最好的规划路线。

2.5 问题五分析

问题五是复杂的多目标规划问题，需要对数据进行处理，以确保符合题目要求。为了优化外国游客在中国游览山景的 144 小时旅游路线，采用 NSGA-II（非支配排序遗传算法 II）这一先进的多目标优化算法来寻找满足游客需求的最佳方案。首先，采用 NSGA-II 算法，生成一个初始种群，其中每个个体代表一个潜在的旅游方案。这些方案随机选择了中国的不同城市（从 352 个可选城市中），并在每个城市中指定了评分最高的山景作为游览目标。同时，这些方案各城市之间的距离规划高铁行程，以确保游客能够在 144 小时内完成所有游览，同时尽量减少总费用和最大化游览的景点数量。接着，算法对种群中的每个解进行评估，计算它们所代表的总景点数量和总费用（包括门票和高铁交通费用），并验证是否满足所有约束条件，如每个城市只游览一座山、总时间不超过 144 小时等。

三、模型假设

- (1) 假设各景点在一天中的任意时刻都可以被游览
- (2) 假设老外一天休息 8 小时，即假设一天 24 小时中有 16 小时可用作旅行
- (3) 假设景点名称中带有“山”字的景点都算作山
- (4) 假设高铁的时速为 300km/h
- (5) 假设高铁票都为二等座
- (6) 假设用欧式距离代表城市间距离

四、符号说明与名称解释

表 1：符号说明

符号	说明
x_{ij}	第 j 个指标的第 i 个样本数值
p_{ij}	归一化处理的比例系数
w_j	第 j 个指标的权重
s_i	第 i 个城市的综合得分

S	问题二获得的前 50 个城市集合
t_{ij}	第 i 个城市到第 j 个城市所需交通时间
T_i	第 i 个城市景点游览时间
S_{ij}	城市间的距离

五、模型的建立与求解

5.1 问题一的模型建立与求解

问题一要求找出所有景点中的最高评分，并找出全国有多少个景点获得了最高评分，以及获得了最高评分的景点有哪些，最后把根据获得最高评分景点数排序出前十的城市，则问题转化为将所有景点数据导入并进行比较、最后保存结果，下面进行模型的建立与求解

5.1.1 模型的建立

Step1:数据预处理:

由于本题是用评分排序，必须有每个景点的评分数据。筛选出附件中空白评分的景点，并在根据提供的网址查找缺失的评分数据后补充进表格，对于查询不到的景点评分填充为无或保留空白，因此数据出现三种形式：有评分、一、无\空白。遇到重复数据时，采用收集到集合中的方法去除重复的数据，如下图评分类型分布：

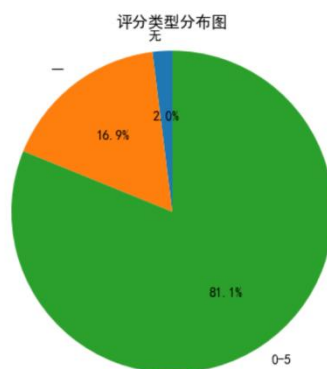


图 1：评分类型分布

Step2:统计比较:

先把各个城市的景点评分数据导入，将一个城市的各景点的评分进行筛选比较，将无效评分(一\无)设为 0，找到每个城市的最高评分景点。然后，对每个城市的最高评分景点数量进行计数，并累计全国范围内获得最高评分的景点数量,最后排出前十名拥有最多最高评分景点数的城市

5.1.2 模型求解

利用 python 进行编程导入全部数据,由程序可以得出全国获得最高评分的景点数量为 2570,并从中发现景点的最高评分为 5.0 分,发现获评最高评分景点数的城市前十位排名如下:

表 2: 前十城市景点数

城市	玉溪	益阳	烟台	潍坊	大兴 安岭	周口	邢台	内江	自贡	雅安
数量	23	21	20	20	18	17	17	17	16	16

5.1.3 可视化分析:

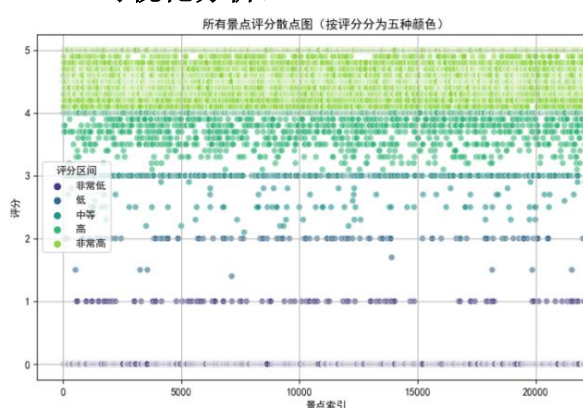


图 2: 35200 各景点评分散点图

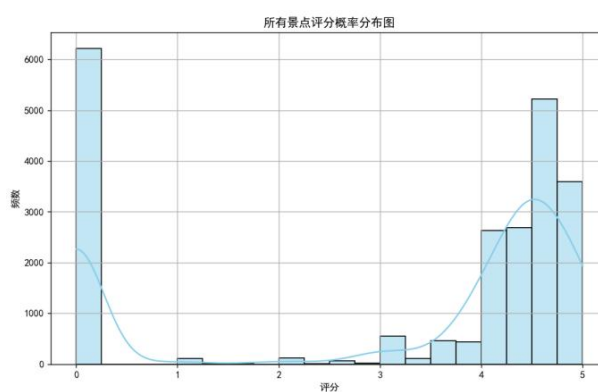


图 3: 所有景点评分概率分布图

由图 2、图 3 可知,大部分景点的评分积分分布在 4-5 分,同时结合图 1 也看到图中部分景点由于评分人过少、没有评分等多种因素而导致 0 分,因此,相关旅游景点应不断完善景区制度同时可以鼓舞游客积极评分,据此做出相应的改善以助力旅游业的蓬勃发展。

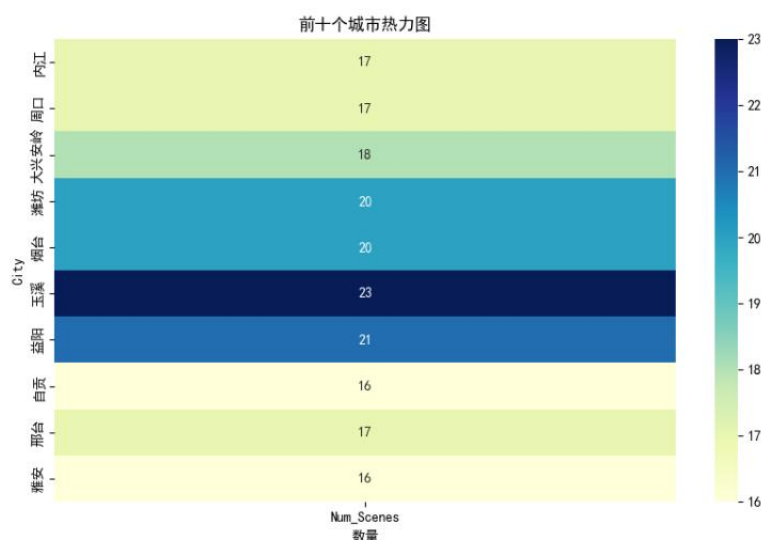


图 4: 前十个城市热力图

图 4 使这十个城市的数量差更明显,只有玉溪获评了最高评分 (BS) 景点最多的城市

5.2 问题二的模型建立与求解

在这一问题中，我们将对 352 个城市进行综合评价，以选出“最令外国游客向往的 50 个城市”。整个方案的逻辑结构与前一个问题相似，但着重于多维指标的综合考虑。目标根据城市规模、环境保护、人文底蕴、交通便利、美食及气候等因素，为每个城市计算综合得分，从而筛选出最理想的 50 个城市，最后，根据本题中提到的“最优景点游览原则”，将筛选出的 50 个城市的所有景点的各项数据利用 python 语言导入，找出它们评分最高的景点，即为“最优景点”，若多个景点为最高分且相同，则利用编程语言随机取出一个，保证公平性。

5.2.1 基于熵权法的多源数据景点评价模型建立：

Step1:数据处理思路：

首先确定评价指标：

评价指标	交通	气候	美食	人文底蕴	环境环保	城市规模
------	----	----	----	------	------	------

其次，利用 Pandas 库从指定文件中读取城市数据，包括多个指标（如高铁线路数量、气候与环保评分等）。其中，高铁线路数据来源于国家铁路局，使用每个城市拥有的高铁线路数量来衡量其交通便利水平。而气候数据来源于各地气象局发布的年度报告中提及的气象灾害的受灾严重程度，其评分越高说明气象灾害越严重，则气候水平越差，因此气候数据为成本型指标（越小越好）。美食业数据来源于国家统计局和各地政府统计局调查的美食行业年度增长率，人文底蕴则采用各地的景点评分之和来表示其人文底蕴是否深厚，因为景点评分的总分数同时与景点数量和景点质量（即评分）有关，此处认为拥有最多景点数和最高景点质量的城市即为人文底蕴最好的城市。城市的环境环保数据使用国家气象局发布的各种污染物污染情况综合评判其环境好坏，所以也为成本型指标。对于城市规模，我们假设人口规模最大的为城市即为规模最大的城市，从国家统计局直接获取相关数据。最后再确保所有相关数值列的数据类型为数值型，处理异常值（如非数字转为 NaN），对于气候和空气污染等负相关指标则进行反向处理。

Step2:具体模型建立

由于各个指标对于外国人是否向往的权重可能不同，而主观给定权重的方式较不严谨，因此考虑使用熵权法对数据进行分析处理，计算各指标的信息熵，以确定每个指标的权重。权重将反映不同指标在综合评价中的重要性。熵权法不仅提高了模型的可靠性，还能自然、客观地处理数据中的权重分配，减少权重分配的主观性，然后根据熵权法得到的权重用以计算每个城市的综合得分。其中熵权法是客观赋权方法中较为常用的一种方法，能够根据数据的变化幅度计算各指标的熵权，能较好地反映各项指标之间的内在逻辑。具体包括以下步骤[1]。

(1) 标准化原始数据

这一步的目的是消除不同指标间的量纲差异，使数据具有可比性，对于第 j 个指标的第 i 个样本，标准化后的值记为 x_{ij}^* 。

若指标为效益型（越大越好），计算公式为：

$$x_{ij}^* = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)}$$

若指标为成本型（越小越好），计算公式为：

$$x_{ij}^* = \frac{\max(x_j) - x_{ij}}{\max(x_j) - \min(x_j)}$$

其中, x_{ij} 表示原始数据矩阵中的元素, $\max(x_j)$ 和 $\min(x_j)$ 分别是第 j 个指标下的最小值和最大值, 对于此问题, 六个指标中, 气候条件是使用受灾损失来衡量, 环境环保是用城市污染程度来衡量, 因此为成本型指标, 其他四个指标都为效益型指标。

(2) 归一化比例系数

这一步的目的为得到每个样本在各个指标下的相对比重, 我们有公式: 对于第 j 个指标的第 i 个样本, 比例系数 p_{ij} 为:

$$p_{ij} = \frac{x_{ij}^*}{\sum_{i=1}^m x_{ij}^*}$$

其中, m 为样本数量, 因此最终这些比例系数应该构成一个矩阵, 在这个公式中, 每个指标 j 下的所有样本 i 的比例系数 p_{ij} 的和为 1。也就是说, 对于每个指标 j , 所有样本的比例系数之和等于 1。这个过程可以看作是一种特殊的归一化处理, 但它主要是为了后续计算熵值做准备

(3) 计算信息熵的值

求出信息熵的目的, 是为了度量各指标的不确定性大小, 对于第 j 个指标, 其信息熵值 e_j 定义为:

$$e_j = -\frac{1}{\ln(m)} \sum_{i=1}^m p_{ij} \ln(p_{ij})$$

其中, $\ln$ 表示自然对数, m 为样本数量

(4) 计算信息效用值

信息效用值反映指标的重要性程度, 熵值越小, 信息效用值越大, 指标越重要。对于第 j 个指标, 其信息效用值 d_j 为:

$$d_j = 1 - e_j$$

其中, e_j 为第 j 个指标的信息熵的值

(5) 计算权重

在算出信息效用值后, 利用信息效用值, 客观判断出各个指标的权重, 对于第 j 个指标, 其权重 w_j 为:

$$w_j = \frac{d_j}{\sum_{j=1}^n d_j}$$

其中 n 为指标的数量。

(6) 计算综合得分

求出各个指标的权重后, 我们便可以利用权重进行加权平均, 求出各个城市的综合得分并进行排名, 筛选出前 50 名的城市, 设第 i 个城市的综合得分为 s_i , 则有以下公式:

$$s_i = \sum_{j=1}^n w_j x_{ij}^*$$

其中 n 为指标的数量。

5.2.3 模型求解

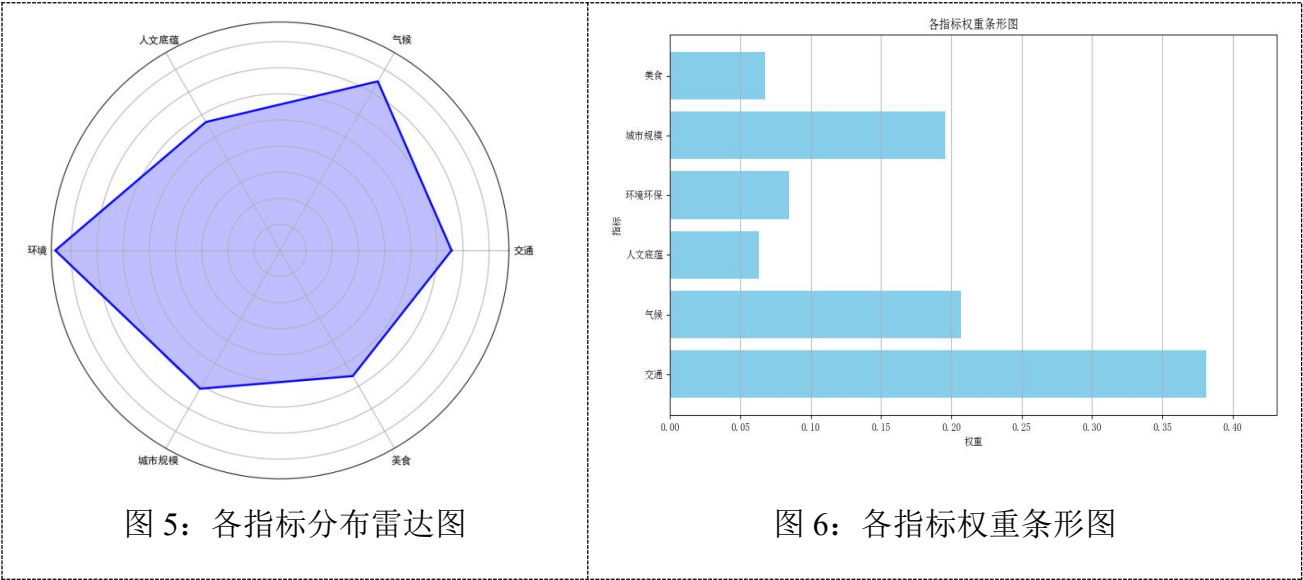
根据上文的公式, 将所有指标各项数据导入, 求出每个指标的各个量如下表:

表 3：各指标数据

指标	交通	气候	人文底蕴	环境 环保	城市 规模	美食
信息熵	0.87247155 67902784	0.93077 0285979 5977	0.978860 64976665 52	0.971 61823 70534 199	0.934 51532 14488 846	0.9773 454156 228655
信息效用值	0.127528443 20972156	0.069229 71402040 234	0.0211393 502333447 9	0.0283 817629 465801 25	0.06548 4678551 1154	0.02265 4584377 134523
权重	0.381343826 66140526	0.20701 5183426 95852	0.063212 25687560 851	0.0848 689893 567263 9	0.1958 165353 379829	0.06774 3208341 31846

汇总各个城市的指标数据，通过加权方式，将标准化后的指标值与对应权重相乘，得到总分。最后选出得分前 50 的城市，并输出相关的指标数据和得分，方便进行分析与比较，并将每个城市的最优景点找出，一并在城市后对应，使得外国游客在游览时能够方便快捷的找出在当前游览原则下应该去的城市以及每个城市该去的最优景点，得出外国人最向往城市及其最优景点。

5.2.4 可视化分析：



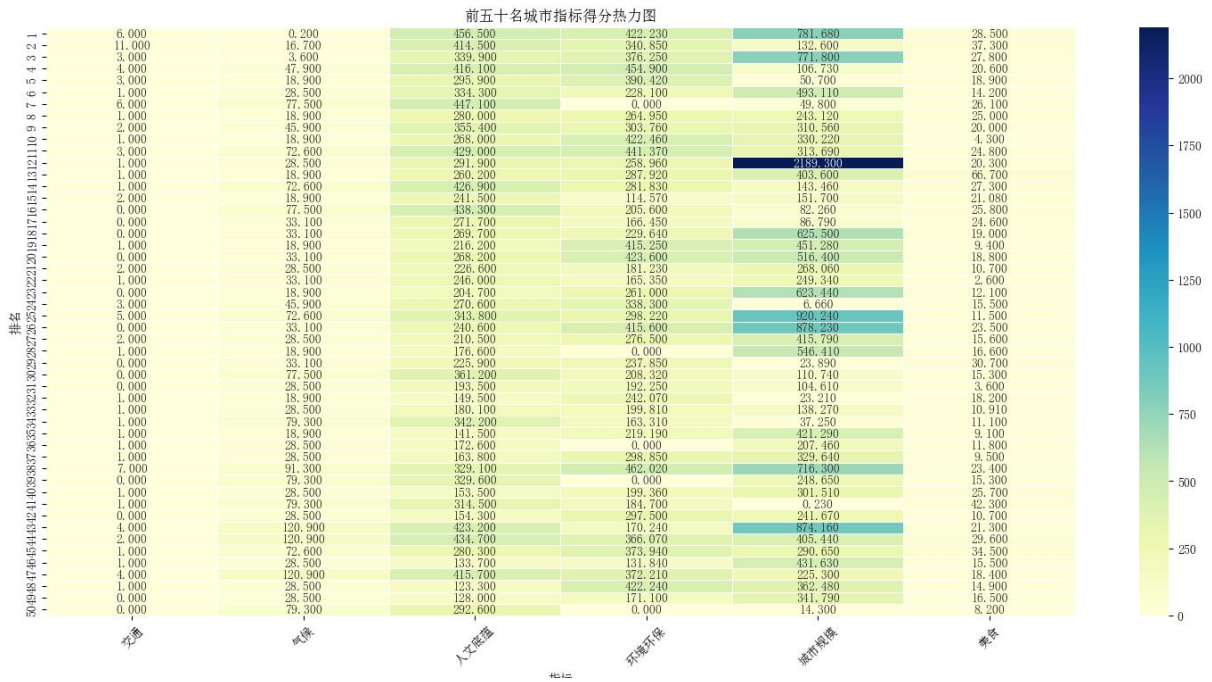


图 7：前五十名城市指标得分热力图

从图 5、图 6 可以清晰得到各指标间的权重，交通的权重占比是最高的，而人文底蕴最少。而图 7 可以看出各城市的美食水平接近，这是因为中国整体上美食行业发展都较为良好，地区间未有重大差异，这样的结论与图 6 中美食的权重占比较小是符合的。而很多城市没有高铁线路，但对于发达城市则可能会具有多条高铁线路，这种差异造成了交通数据的差异较大，从而导致其权重大，也同样能够在图 6 中得到体现。

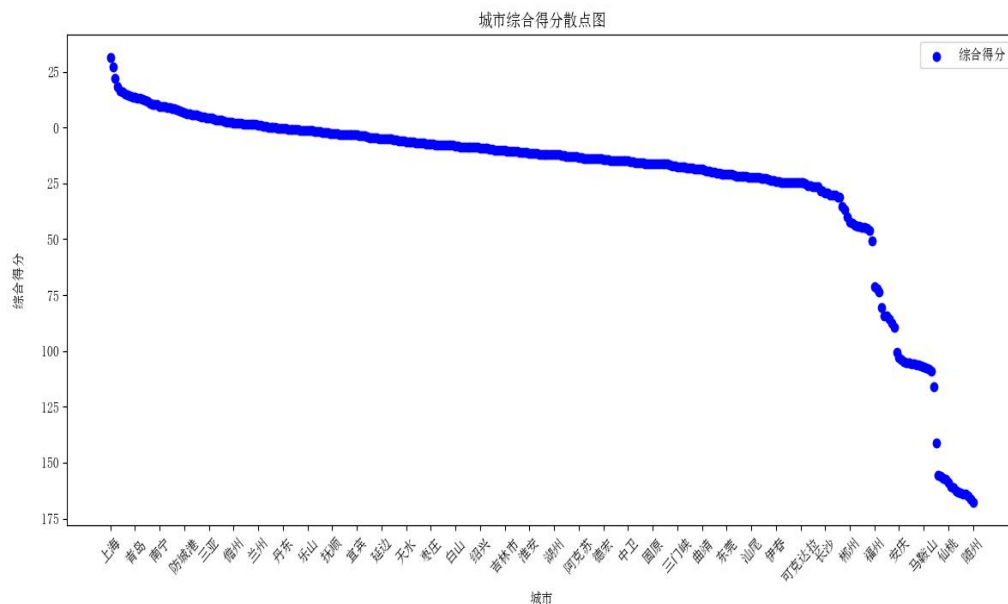


图 8：城市综合得分散点图

从图 8 则可以看出，前五十名城市的得分情况呈现出，高分密集，剩下的城市则零散分布在下方，差距也相对更大。

5.3 问题三的模型建立与求解:

这一问题可以视为一个多目标优化问题，重点在于在规定的时间内（除去睡眠时间后的游玩时间 96 小时）最大化游览城市数量，同时最大化游玩体验。其中为了方便建立约束，将规定时间转化为分钟。本文使用混合整数线性规划（MILP）模型来处理这个外国游客的游玩路线规划问题是一个相对复杂的任务，因为 MILP 主要用于优化问题，其中涉及到线性目标函数、线性约束和整数变量。然而，这个问题包含了许多非线性和复杂的因素，如不同城市的景点综合评分、高铁的票价和时间等，这些都难以直接转化为 MILP 模型的标准元素。不过，可以尝试将这个问题简化和抽象化，以便能够构建一个近似的 MILP 模型。其中为了方便建立约束，将规定时间转化为分钟。以下是模型的详细建立与求解过程：

5.3.1 混合复杂因素规划路径模型建立

Step1:数据准备:

收集所需的输入数据，包括所有城市的综合评分 s_i (问题二中获得)、城市间高铁交通时间 t_{ij} （由于每天每趟的时间都不同，团队统一在铁路 12306 中查询后采取 8.7 号的平均值）和游玩城市所需的参观时间 T_i （附件中获得，附件中空白的则从大众点评网中获取取中值）。确认“最令外国游客向往的 50 个城市”集合 S，确保只有这些城市的选择被考虑。

Step2:模型建立:

我们需要设计如下的目标函数与约束条件：

(1) 目标函数:

我们的目标是最大化游客在游玩的城市中获得的综合体验评分。设 s_i 表示城市 C_i 的综合评分，则目标函数可以表示为：

$$\text{Maximize } Z = \sum_{i=1}^n s_i \cdot x_i$$

其中， x_i 是游玩城市 C_i 的决策变量，若选择该城市则 $x_i=1$ ，否则为 0。

(2) 时间约束:

为确保游客在规定时间内游玩，我们将总游玩时间约束设定为：

$$\sum_{i=1}^n (t_{ij} + s_i) \cdot x_i \leq T$$

其中， t_{ij} 表示游客在城市 C_i 与城市 C_j 之间的高铁交通时间， s_i 为游玩城市 C_i 所需的参观时间。

(3) 城市选择约束:

为了确保游客选择的城市中只包含在“最令外国游客向往的 50 个城市”之内，我们有：

$$x_i \in \{0,1\} (i=1,2,\dots,n)$$

其中，只有在集合为游玩的城市仅限于外国游客向往的 50 个城市集合 S，我们有：

$$x_i=0 \text{ if } c_i \in /S$$

(4) 非负约束:

确保所有决策变量为二进制值:

$$x_i \in \{0,1\} (i=1,2,\dots,n)$$

5.3.2 模型求解

在模型建立完成后，接下来需要对该混合整数线性规划（MILP）模型进行求解。

以下是求解过程的详细步骤:

(1) 数据准备:

收集所需的输入数据，包括所有城市的综合评分 s_i 、城市间高铁交通时间 t_{ij} 和游玩城市所需的参观时间 s_i 。确认“最令外国游客向往的 50 个城市”集合 S ，确保只有这些城市的选择被考虑。

(2) 建模实现:

在所选的 python 环境中，使用求解器的 API 构建模型：创建变量 x_i ，表示决策变量（选择城市）。定义目标函数：最大化游客获得的综合体验评分。添加时间约束，确保总游玩时间不超过规定的 T 。添加城市选择约束，确保只选择允许的城市 (J 即集合 S)。

(3) 求解结果:

总游玩时间: 5745 分

总费用: 3975 元

总游玩景点数量: 39 个

游玩城市列表: ['张家口', '烟台', '邢台', '五指山', '无锡', '潍坊', '唐山', '苏州', '石家庄', '沈阳', '上海', '山南', '三亚', '日喀则', '青岛', '秦皇岛', '钦州', '南宁', '那曲', '柳州', '陵水', '林芝', '廊坊', '拉萨', '金华', '济南', '衡水', '桂林', '贵港', '甘孜', '防城港', '崇左', '承德', '成都', '北京', '北海', '保定', '阿里', '阿坝']

5.3.3 可视化分析:

运用 Pyecharts 库展示路线数据。以下是分析的具体步骤和逻辑:

1. 数据准备: 首先，选择模型求解结果中题目的城市。
2. 坐标统一: 为了在地图上准确定位，手动定义了每个城市的经纬度坐标。此举确保在后续的可视化过程中，每个城市能够准确地显示在相应的位置。
3. 创建地图: 使用 Geo 图表，以中国地图为基础，设置地图类型为“china”，并为每个城市添加相应的坐标数据。
4. 数据可视化: 对每个城市，使用了 EFFECT_SCATTER 类型来标记其位置。通过这种方式，可以直观地看到路线涉及的所有城市。另外，定义了一条旅游路线，连接城市之间的线段，使用 LINES 类型进行展示。为增强可视化效果，在线条上添加了箭头标识，显示旅游的方向。
5. 参数设置: 为了提升可视化效果，设置了线条的样式与颜色，包括使用蓝色线条和透明度参数，以便更好地区分和理解路径。
6. 最终展示: 通过设置标题和视觉映射选项，确保最终的地图图表既美观又实用，最终结果输出为 HTML 文件，使得可视化分析便于分享与展示。最终得到旅游路线图如图 9 所示:

问题3路线图

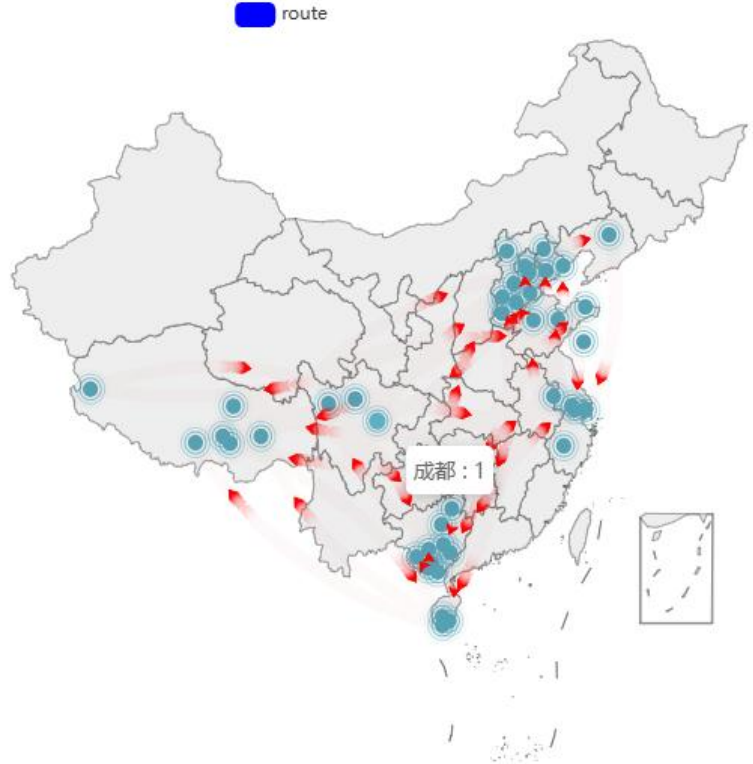


图 9：问题 3 路线图

5.4 问题四的模型建立与求解

问题四与问题三类型相同，要求改变为在尽可能游览更多城市的同时，控制门票和交通的总费用。这一问题同样可以视为一个多目标优化问题，重点变为在规定的时间内（除去睡眠时间后的游玩时间 96 小时）最大化游览城市数量，同时最小化费用。其中为了方便建立约束，将规定时间转化为分钟。以下是模型的详细建立与求解过程。

5.4.1 目标函数与约束条件的建立

定义集合 $C=\{c_1, c_2, \dots, c_n\}$ 为 50 个目标游玩的城市，城市呈现为 c_1 到 c_n 。每个城市的游玩所需时间和费用是决策的重要因素。具体如下：

游玩时间： t_i ： 在城市 c_i 中的总游玩时间，包括交通时间和景点游玩时间。

费用： p_i ： 城市 c_i 的交通与门票费用之和。

基于以上分析，目标函数与约束条件的构建如下：

（1）目标函数： 需要最大化游玩城市数量并最小化费用。设计的复合目标函数如下所示：

$$\max Z = \sum_{i=1}^n x[i] - \lambda \sum_{i=1}^n (P_i \cdot x[i])$$

其中， $x[i]$ 为决策变量，表示城市 c_i 是否被选择游玩， λ 是权重系数，用于调整两者的权衡。

(2) 时间限制约束：为保证游玩时间控制出去睡眠时间的游玩时间内，定义时间约束：

$$\sum_{i=1}^n lt_j \cdot x[i] \leq T$$

(3) 城市选择约束：为了确保至少游玩一个城市，需要如下约束：

$$\sum_{i=1}^{\eta} x[i] \geq 1$$

5.4.2 模型求解

在新的约束条件确定、目标函数确定后，模型建立完成，接下来需要对这个新的混合整数线性规划模型进行求解。以下是求解过程的详细步骤：

(1)数据准备：

与问题三的求解过程类似的，先收集所需的输入数据，包括各个城市的景点门票和交通费用 p_i 、城市间高铁交通时间 t_{ij} 和游玩景点所需的参观时间 s_i 。确认“最令外国 游客向往的 50 个城市”集合 S 中的元素，确保只有这些城市可被选择。

(2)建模实现求解过程：

在与之前相同的 `python` 编译环境中，使用求解器构建模型：创建变量 x_i ，表示决策变量（选择城市），变量的 0 或 1 代表是否游玩该城市。定义复合目标函数：最大化游玩城市数量并最小化费用。添加时间约束，确保总游玩时间不超过规定的 T 。添加城市选择约束，确保只选择允许的 50 个城市。

(3)求解结果：

总游玩时间: 5745 分

总费用: 3830 元

总游玩景点数量: 39 个

游玩城市列表: ['张家口', '玉林', '烟台', '邢台', '五指山', '无锡', '潍坊', '唐山', '苏州', '石家庄', '沈阳', '上海', '山南', '三亚', '日喀则', '青岛', '秦皇岛', '钦州', '南宁', '南京', '那曲', '柳州', '陵水', '林芝', '拉萨', '金华', '济南', '衡水', '桂林', '贵港', '甘孜', '防城港', '崇左', '承德', '成都', '北京', '北海', '保定', '阿坝']

5.4.3 可视化分析

本次可视化分析的目的在于展示一条覆盖多个城市的旅游路线，以便使用能够直观地了解路线布局及各城市之间的关系。具体步骤同问题 3，最终旅游规划路线图如下：

问题4路线图

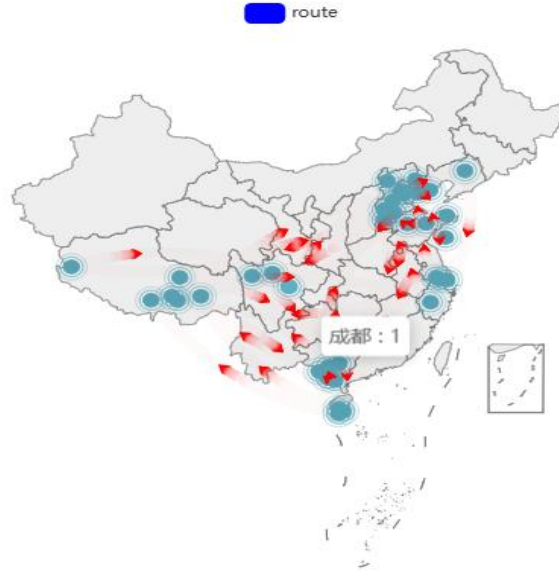


图 10: 问题 4 路线图

5.5 问题五的模型建立与求解

建立多目标规划遗传算法模型来求解旅行计划问题，旨在找到一条旅行路线，使得在给定的时间限制下，访问尽可能多的城市，并尽量减少旅行费用。该方法的建立过程如下：

5.5.1 基于 NSGA-II 的多冲突目标线路规划模型建立：

Step1: 数据准备：

利用 python 筛选出含有山的城市以及相应的评分最高的山，同时，为确保筛选出的城市有高铁线路，删去了没有高铁线路的城市。为方便随意两个相邻城市间距离、费用、交通时间的计算以及可视化线路图，团队搜集了这些城市的经纬度。为简化问题，我们将所有城市的高铁时速、票价都看为一致，据铁路客运运价里程表等公告综合评估时速 300 km/h 及以上的线路，G 字头列车（以下简称 G 车）二等座票价率一般为 0.46 元/公里。

距离计算公式：

$$d = \sum_{i,j}^m \sqrt{(l_i - l_j)^2 + (q_i - q_j)^2}$$

Step2: 模型建立：

(1) 初始化种群[4]

初始化种群涉及到创建一个包含多个候选解（称为“个体”）的初始集合。通常使用二进制编码、整数编码或其他编码方式来表示解。在创建初始种群时随机生成一定数量的个体来构成初始种群，假设我们使用整数编码表示解，并且问题涉及访问一系列城市，每个个体表示一个旅行顺序，则可以表示为：

$$\text{个体} = [i_1, i_2, \dots, i_n]$$

其中 i_j 是第 j 个城市编号。此处使用 `toolbox.population(n=100)` 创建了一个包含 100 个个体的初始种群。每个个体由 `toolbox.indices` 生成，确保满足每个城市只访问一个景点的条件，这里使用了 `np.random.permutation` 来生成一个随机排列的城市索引列表。

(2) 定义适应度函数

需考虑两个主要目标：最小化总费用（包括门票和交通）和最大化游览的山景数量。
目标：

$$\begin{cases} \max m \\ \min \sum_{i,j}^m (x_i + x_{i,j}) \end{cases}$$

限制条件：

$$\sum_{i,j}^m (t_i + \Delta t_{i,j}) \leq 66$$

适应度函数用于评估每个个体的优劣程度，应根据问题的特点来设计，用于衡量个体的质量，其公式如下：

$$F(\text{个体}) = w_1 * \text{Cost}(\text{个体}) + w_2 * \text{Time}(\text{个体}) - w_3 * \text{NumCities}(\text{个体})$$

其中 w_1, w_2, w_3 是权重系数，用于平衡三个目标的重要性，`Cost`, `Time`, `NumCities` 分别是旅行费用、旅行时间和访问的城市数量。

(3) 选择

使用 NSGA-II 的非支配排序和拥挤距离来选择个体，确保种群的多样性和收敛性。

(4) 交叉

交叉操作通过组合两个父代个体的部分信息来产生新的子代个体，常见的交叉类型包括单点交叉、多点交叉、均匀交叉等，此处使用的是有序交叉，有序交叉 (`cxOrdered`) 是一种专门用于排列问题的交叉方法。它确保后代个体的城市序列仍然是有效的。

(5) 变异

变异操作用于在个体中引入随机变化，增加种群的多样性。常见的变异类型包括位翻转变异、交换变异等，而此处采用了序列中的索引互换变异，索引互换变异 (`mutShuffleIndexes`) 是一种用于排列问题的变异方法，它通过随机交换城市序列中的两个元素位置来实现变异。

(6) 替换

替换操作用于决定哪些个体进入下一代种群，常见的替换策略包括精英保留策略、完全替换策略等，此处我们使用了 DEAP 的 `eaMuPlusLambda` 算法来实现替换，DEAP 库中的 `eaMuPlusLambda` 算法是基于 $\mu + \lambda$ 进化策略的一个实现。这种进化策略是一种常用的遗传算法变体，它结合了选择、交叉和变异操作来产生下一代种群。 $\mu + \lambda$ 进化策略的工作流程如下：从当前种群中选择 μ 个个体作为父母个体，通过交叉和变异操作从 μ 个父母个体产生 λ 个后代个体，将 μ 个父母个体与 λ 个后代个体合并成一个新的种群，种群大小为 $\mu + \lambda$ 。从 $\mu + \lambda$ 个个体中选择 μ 个个体作为下一代种群。重复以上步骤，直到达到预定的终止条件。[5]

(7) 终止条件

终止条件用于判断算法何时停止，常见的终止条件包括达到最大迭代次数、适应度值不再改变等，终止条件用于判断算法何时停止。在您的代码中，终止条件是进化代数（ngen），ngen=50 表示算法运行 50 代后停止。

(8) 结果分析与选择

重复执行选择、交叉、变异和替换操作，直到满足终止条件，此处我们在满足终止条件后，评估最佳个体，获取最佳路线、访问的城市数量、总时间和总费用。

5.5.2 模型求解：

将数据导入后，模型通过导入的位置数据采用欧式距离计算，并进一步计算出费用后，进行多目标 NSGA-II 优化，得到总时间约为 66 小时，总费用约为 58396 元，路线如下表：

表 4 路线表格

城市	景点	总时间 (小时)	总费用 (元)
秦皇岛	金山嘴	66	5839.5 9946
开封	山峡甘会馆		
牡丹江	山庄码头		
烟台	艾山		
丹东	虎山长城-三号空心敌楼		
玉溪	金山原始森林		
泉州	梁山樱花观赏园		
珠海	四大佛山旅游风景区		
东莞	东莞龙凤山影视度假村		
中山	云梯山花海		
滁州	神山森林公园		
深圳	航佳绿色生态家园度假山庄		

5.5.3 数据可视化分析：



结合图 12 高铁的线路分布明显会影响到图 11 游览城市的筛选，高铁线路分布多的地区共有游览的城市也偏多

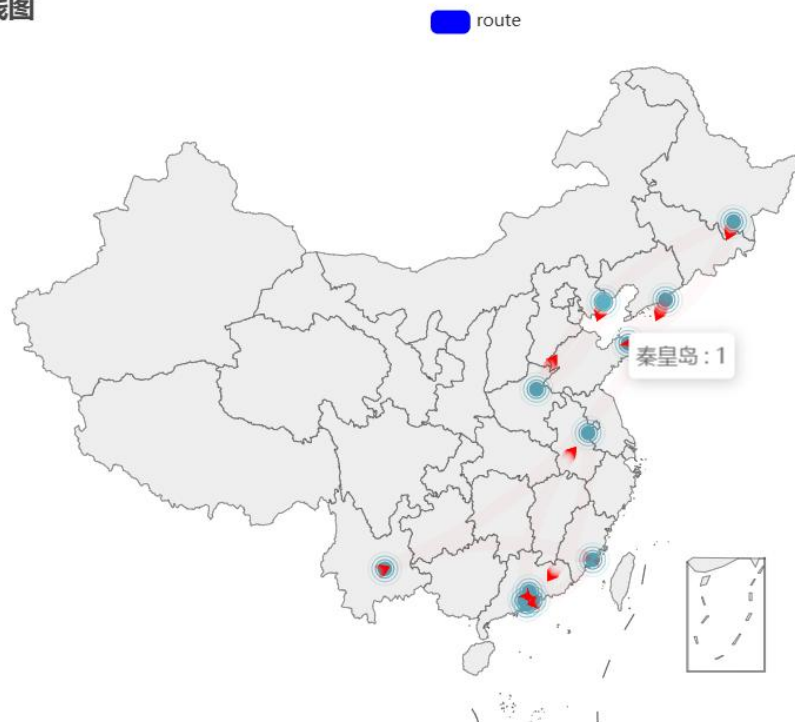


图 13：最优路线图

根据图 13 更直观的看到最优路线的分布，

六、模型的评价与推广

6.1 模型的优点

- 问题一在对数据进行处理时做到了逻辑清晰，对错误考虑周到：使用了 python 中的 Pandas 库来读取 CSV 文件，并对评分列进行了数值类型的转换和处理，保证了数据的一致性。并通过使用 `pd.to_numeric()` 函数和 `.fillna()` 方法，对无法转换的评分值进行了处理，避免了由此引起的程序异常。

问题二本模型通过熵权法综合考虑了多个影响因素，如城市规模、环境、交通、人文底蕴等，从而有效地评估 352 个城市的综合吸引力。这种方法能够在考虑多元化因素的同时，确保评分的客观性与科学性。在计算权重方面，熵权法能够自适应地反映各指标的信息贡献度，通过信息熵的方式合理分配权重，有助于避免主观偏见，提高模型的可靠性。模型通过标准化和归一化处理，使得不同量纲的指标能够统一纳入综合评分中，增强了分析结果的可比性。同时，反向处理负相关指标（如空气污染）有效提升了模型的实用性，真实反映了游客的偏好。

问题三、四都被转化为了多目标的线性规划问题，只是两个问题的目标函数和约束条件稍有改动，因此它们的优缺点类似。其中代码编写都采用了模块化的结构使得每个部分的功能明确，易于理解和维护。变量命名直观，如 `x` 代表是否选择某个城市，使读者能够快速理解其含义和作用。而后通过线性规划求解器（SCIP），代码有效地优化了旅行计划，在给定的时间限制内最大化游玩体验。通过设置时间限制作为约束条件，

确保了旅行计划既充实又合理。代码易于调整时间限制和其他参数，可以根据不同的旅行需求进行定制。此外，还可以方便地添加额外的约束条件或目标函数，以适应更复杂的需求场景，有较好的可扩展性。对于最终输出的结果，包括总游玩时间、总费用、游玩景点数量以及游玩城市列表，这些信息对旅行规划者来说非常有用。结果以清晰明了的方式呈现，便于旅行者根据结果制定详细的行程安排。

问题五基于 NSGA-III 的多冲突目标线路规划的模型可以同时优化总时间、总费用和访问城市数量，这在实际旅行规划中很实用。

6.2 模型的缺点：

问题一中的代码假设所有 CSV 文件都具有相同的列名和数据结构，对于不同结构的 CSV 文件可能无法正常处理，因此模型的复用性较差，当需要在其他地方用到时，需要的修改较多，在处理大量文件或大型 CSV 文件时也可能存在效率较低

问题二在数据预处理阶段，对于缺失值的处理虽使用了填充方法，但仍可能影响最终结果的准确性。某些指标如果数据量过小或缺失值过多，模型评估的可靠性可能下降。虽然熵权法能自适应权重分配，但在指标呈现高度相关性时，可能导致权重分配的不均衡，这样会影响到城市综合得分的准确性。模型未能考虑潜在的交互效应，例如环境因素与人文底蕴之间的关系，这可能导致某些城市的真实吸引力未能充分体现。

问题三、四的求解过程中，都没有时间限制的设定，虽然代码中设定了时间限制作为约束条件，但并没有为求解器设置最大运行时间，这可能导致求解器无限期运行直到找到最优解。在实际应用中，应为求解器设置一个合理的最大运行时间，以防止无限期运行。且最终的输出结果仍需要一定的优化，虽然代码提供了游玩时间、费用、景点数量和游玩城市列表等基本信息，但对于更详细的信息则没有给出，如每个城市的预计到达时间和离开时间。

6.3 模型的推广

问题二在数据预处理时，可以采用更为细致的缺失值处理策略，如多重插补，确保数据的完整性，从而提高模型评估的准确性。进一步完善熵权法，通过引入主成分分析（PCA）来降低指标之间的多重共线性，提高权重的稳定性与合理性，并为每项指标提供更具解释力的结果。考虑引入定性评价方法或专家咨询，以补充模型中未能反映的城市吸引力要素，尤其是城市独特文化与旅游体验的影响，为综合评价提供更多维度的信息。

在问题三和四的求解过程中，可以考虑为求解器设置最大运行时间，以防止无限期运行直到找到最优解。这可以通过 `solver.SetTimeLimit(30000)` 语句设置求解器最大运行时间为 30 秒的方式来实现。除此之外，除了当前的基本信息（如总游玩时间、总费用、游玩景点数量和游玩城市列表）之外，还可以提供更详细的旅行规划信息，如每个城市的预计到达时间和离开时间了，使老外能够清楚明了的看出此次旅行的规划。

基于此问题五模型扩展到其他旅行规划问题，如旅游路线优化、物流配送优化等，只需调整适应度函数和城市数据的输入格式即可。

七、参考文献

- [1] 李旭, 李景文, 俞娜. 基于用户需求的旅游路线推荐方法[J]. 计算机工程与设计, 2021, 42(05):1339-1345. DOI:10.16208/j.issn1000-7024.2021.05.020.
- [2] 熊搏. 基于动态规划的旅游路径算法研究[D]. 海南大学, 2023. DOI:10.27073/d.cnki.ghadu.2023.000113.
- [3] 邓俊, 韦化, 黎静华, 白晓清, 一种含四类 0-1 变量的机组组合混合整数线性规划模型, 广西大学, 2015, DOI: 10.13334/j.0258-8013.pcsee.2015.11.015.
- [4] 王远飞. 基于遗传算法的水质模型参数估计研究[J]. 市政技术, 2024, 42(08):205-214+244.
- [5] 许向荣, 基于改进遗传算法与多源异构数据的旅游推荐算法研究, 西安理工大学, 2024, DOI:10.27398/d.cnki.gxalu.2023.001863:9-10, 2024.

```

附录：
# 问题一
import os
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
base_dir = r'H:\数学建模\华数杯\解C题\地点'
files = [os.path.join(base_dir, file) for file in os.listdir(base_dir) if
file.endswith('.csv')]
max_score = -float('inf')
max_num = 0
top_sites_dict = {}
for file_path in files:
    df = pd.read_csv(file_path, encoding='utf-8')
    if '评分' in df.columns and '名字' in df.columns:
        df['评分'] = pd.to_numeric(df['评分'], errors='coerce').fillna(0)
        current_max_score = df['评分'].max()
        if current_max_score > max_score:
            max_score = current_max_score
        top_sites = df[df['评分'] == max_score]['名字'].dropna().unique()
        num_scenes = len(top_sites)
        max_num += num_scenes
        city_name = os.path.basename(file_path).replace('.csv', '')
        if city_name in top_sites_dict:
            top_sites_dict[city_name] += num_scenes
        else:
            top_sites_dict[city_name] = num_scenes
print(f"全国最高评分为: {max_score}")
print(f"全国最高评分的景点数量: {max_num}")
sorted_all_files = sorted(top_sites_dict.items(), key=lambda x: x[1],
reverse=True)
top_ten_cities = sorted_all_files[:10]
print("前十的城市如下: ")
for i, (city_name, num_scenes) in enumerate(top_ten_cities, 1):
    print(f"{city_name}: {num_scenes}")
cities = [city for city, _ in top_ten_cities]
num_scenes = [num for _, num in top_ten_cities]
data = pd.DataFrame({'City': cities, 'Num_Scenes': num_scenes})
plt.figure(figsize=(10, 6))
sns.heatmap(data.pivot_table(index='City', values='Num_Scenes',
aggfunc='sum'), annot=True, cmap='YlGnBu', fmt='.0f')
plt.title('前十个城市热力图')

```

```

plt.xlabel('数量')
plt.ylabel('城市')
plt.show()
all_scores = []
for file_path in files:
    df = pd.read_csv(file_path, encoding='utf-8')
    if '评分' in df.columns:
        scores = pd.to_numeric(df['评分'], errors='coerce').dropna()
        all_scores.extend(scores)
plt.figure(figsize=(10, 6))
sns.histplot(all_scores, bins=20, kde=True, color='skyblue')
plt.title('所有景点评分概率分布图')
plt.xlabel('评分')
plt.ylabel('频数')
plt.grid(True)
plt.show()

# 问题二
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimSun', 'Arial']

file_path = r'C:\Users\12621\Desktop\徐泽民\数学建模\问题二数据\问题二数据.csv'
data = pd.read_csv(file_path, encoding='utf-8')

numeric_columns = ['交通', '气候', '人文底蕴', '环境环保', '城市规模', '美食']
for col in numeric_columns:
    data[col] = pd.to_numeric(data[col], errors='coerce')

def entropy_weight(df, columns):

    entropy_list = []
    information_utility_list = []
    for col in columns:
        p = df[col] / df[col].sum()
        entropy = -np.sum(p * np.log(p + 1e-10)) / np.log(len(p))
        information_utility = 1 - entropy
        entropy_list.append(entropy)
        information_utility_list.append(information_utility)

    weights = [(1 - e) / (len(columns) - np.sum(np.isnan(entropy_list)))] for e in entropy_list

```

```

weights_normalized = np.array(weights) / np.sum(weights)

return weights_normalized, entropy_list, information_utility_list

def calculate_scores(df, columns, air_pollution, resident_population):
    air_pollution = air_pollution.fillna(0)
    resident_population = resident_population.fillna(0)
    weights, entropies, information_utilities = entropy_weight(df, columns)

    for i, col in enumerate(columns):
        df[f'{col} 权重'] = weights[i]
        df[f'{col} 熵'] = entropies[i]
        df[f'{col} 信息效用值'] = information_utilities[i]

    normalized_values = df[columns].copy()

    normalized_values['气候'] = 1 - normalized_values['气候']

    normalized_values['环境环保'] = 1 - air_pollution / air_pollution.max()

    normalized_values['城市规模'] = resident_population / resident_population.max()

    normalized_values['美食'] = df['美食'] / df['美食'].max()

    scores = np.sum(normalized_values * weights, axis=1)

    return scores

air_pollution_scores = data['环境环保']
resident_population = data['城市规模']

food_scores = data['美食']

if len(food_scores) != len(data):
    raise ValueError("美食数据列的长度与 DataFrame 的行数不一致。")

data['美食'] = food_scores

indicators = ['交通', '气候', '人文底蕴', '环境环保', '城市规模', '美食']

weights, entropies, information_utilities = entropy_weight(data, indicators)
for i, col in enumerate(indicators):
    print(f'{col} 的权重为： {weights[i]}, 熵为： {entropies[i]}, 信息效用值为：

```

```

{information_utilities[i]})

data[' 综 合 得 分 '] = calculate_scores(data, indicators, air_pollution_scores,
resident_population)
top_cities = data.nlargest(50, '综合得分')

print(top_cities[['地区', '交通', '气候', '人文底蕴', '环境保护', '城市规模', '美食', '综合得分'
]])

print("所以最令外国游客向往的 50 个城市分别为: ")
for idx, city in enumerate(top_cities['地区']):
    print(f'{idx + 1}. {city}')
data = data.sort_values(by='综合得分', ascending=False).reset_index(drop=True)

top_50_cities = data.head(50)

import matplotlib.pyplot as plt

data = data.sort_values(by='地区')

plt.figure(figsize=(12, 6))
plt.scatter(data['地区'], data['综合得分'], marker='o', color='b', label='综合得分')
plt.xticks(range(0, len(data['地区']), 10), data['地区'].iloc[::10], rotation=45)
plt.xlabel('城市')
plt.ylabel('综合得分')
plt.title('城市综合得分散点图')
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 10))
sns.heatmap(top_50_cities[indicators], annot=True, fmt='.3f', cmap='YlGnBu', linewidths=.5)
plt.title('前五十名城市指标得分热力图')
plt.xlabel('指标')
plt.ylabel('排名')
plt.xticks(rotation=45)
plt.yticks(np.arange(0.5, 50.5, 1), range(1, 51))
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))
plt.barh(indicators, weights, color='skyblue')
plt.xlabel('权重')

```



```

plt.ylabel('指标')
plt.title('各指标权重条形图')
plt.xlim(0, max(weights) + 0.05)
plt.grid(axis='x')
plt.tight_layout()
plt.show()

```

```

print("前五十名城市的指标得分热力图和城市综合得分散点图还有权重图分别如上所示。")

```

问题三

```

import pandas as pd
from ortools.linear_solver import pywraplp

top_50_cities = pd.read_csv(r"D:\B\问题 3、4 数据.csv", encoding='GBK')

if top_50_cities is None :
    print("某些数据未能成功读取，请检查文件路径和编码。")
    exit()

solver = pywraplp.Solver.CreateSolver('SCIP')

cities = top_50_cities['城市'].tolist()
n = len(cities)
x = {i: solver.IntVar(0, 1, f'x[{i}]')} for i in range(n)}

travel_time = top_50_cities.set_index('城市')['交通时间'].to_dict()
sight_time = top_50_cities.set_index('城市')['参观时间'].to_dict()
sight_cost = top_50_cities.set_index('城市')['门票费用'].to_dict()
city_score = top_50_cities.set_index('城市')['综合评分'].to_dict()
travel_cost = top_50_cities.set_index('城市')['交通费用'].to_dict()

time_limit = 5760 # 分
time_constraints = solver.Constraint(0, time_limit)
for city in cities:
    time_constraints.SetCoefficient(x[cities.index(city)],
    travel_time.get(city, 0) + sight_time.get(city, 0))

objective = solver.Objective()
for city in cities:
    objective.SetCoefficient(x[cities.index(city)], city_score.get(city, 0))
objective.SetMaximization()

```

```

status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    total_time = sum(travel_time.get(cities[i], 0) + sight_time.get(cities[i],
0) for i in range(n) if x[i].solution_value() == 1)

    total_cost = sum(sight_cost.get(cities[i], 0) + travel_cost.get(cities[i],
0) for i in range(n) if x[i].solution_value() == 1)
    total_sights = sum(1 for i in range(n) if x[i].solution_value() == 1)
    visited_cities = [cities[i] for i in range(n) if x[i].solution_value() ==
1]

    print("总游玩时间:", total_time, "分")
    print("总费用:", total_cost, "元")
    print("总游玩景点数量:", total_sights, "个")
    print("游玩城市列表:", visited_cities)
else:
    print("没有找到最优解。")
# 问题三路线图
from pyecharts import options as opts
from pyecharts.charts import Geo
from pyecharts.globals import ChartType, SymbolType

cities = ['张家口', '玉林', '烟台', '邢台', '五指山', '无锡', '潍坊', '唐山', '苏州',
          '石家庄', '沈阳', '上海', '山南', '三亚', '日喀则', '青岛', '秦皇岛', '钦州',
          '南宁', '南京', '那曲', '柳州', '陵水', '林芝', '廊坊', '拉萨', '金华', '济南',
          '衡水', '桂林', '贵港', '甘孜', '防城港', '崇左', '承德', '成都', '沧州', '北京', '北海', '
保定', '阿里', '阿坝']

coordinate_override = {
    '张家口': [114.8875, 40.8244],
    '玉林': [110.1647, 22.6332],
    '烟台': [121.4479, 37.4645],
    '邢台': [114.5048, 37.0706],
    '五指山': [109.5169, 18.7751],
    '无锡': [120.3119, 31.4912],
    '潍坊': [119.1618, 36.7068],
    '唐山': [118.1802, 39.6305],
    '苏州': [120.5832, 31.2983],
    '石家庄': [114.5149, 38.0428],
    '沈阳': [123.4315, 41.8057],
    '上海': [121.4737, 31.2304],

```

```

    '山南': [91.7714, 29.2286],
    '三亚': [109.5119, 18.2528],
    '日喀则': [88.8818, 29.2671],
    '青岛': [121.3119, 35.3512],
    '秦皇岛': [119.5996, 39.9354],
    '钦州': [108.6541, 21.9797],
    '南宁': [108.3669, 22.8170],
    '南京': [118.7969, 32.0603],
    '那曲': [92.0513, 31.4761],
    '柳州': [109.4318, 24.3246],
    '陵水': [110.0375, 18.5050],
    '林芝': [94.3497, 29.6516],
    '廊坊': [116.6997, 39.5128],
    '拉萨': [91.1721, 29.6525],
    '金华': [119.6474, 29.0791],
    '济南': [117.1201, 36.6512],
    '衡水': [115.6702, 37.7389],
    '桂林': [110.2991, 25.2742],
    '贵港': [109.5989, 23.1133],
    '甘孜': [99.9927, 31.6190],
    '防城港': [108.3490, 21.6930],
    '崇左': [107.3649, 22.3774],
    '承德': [117.9634, 40.9515],
    '成都': [104.0668, 30.5728],
    '沧州': [116.8388, 38.3044],
    '北京': [116.4074, 39.9042],
    '北海': [109.1201, 21.4811],
    '保定': [115.4646, 38.8739],
    '阿里': [80.1055, 32.5011],
    '阿坝': [102.2247, 31.8998],
}

route = ['张家口', '玉林', '烟台', '邢台', '五指山', '无锡', '潍坊', '唐山', '苏州',
        '石家庄', '沈阳', '上海', '山南', '三亚', '日喀则', '青岛', '秦皇岛', '钦州',
        '南宁', '南京', '那曲', '柳州', '陵水', '林芝', '廊坊', '拉萨', '金华', '济南',
        '衡水', '桂林', '贵港', '甘孜', '防城港', '崇左', '承德', '成都', '沧州', '北京', '北海', '
保定', '阿里', '阿坝']

geo = Geo()

geo.add_schema(maptype="china")

for city in cities:
    if city in coordinate_override:

```

```

        geo.add_coordinate(city, coordinate_override[city][0], coordinate_override[city][1])
    geo.add("", [(city, 1)], type_=ChartType.EFFECT_SCATTER, symbol_size=10)

geo.add(
    "route",
    [(route[i], route[i+1]) for i in range(len(route)-1)],
    type_=ChartType.LINES,
    effect_opts=opts.EffectOpts(symbol=SymbolType.ARROW, symbol_size=6,
    color="red"),
    linestyle_opts=opts.LineStyleOpts(curve=0.2, color="blue", opacity=0.7, width=2),
)

geo.set_series_opts(label_opts=opts.LabelOpts(is_show=False))
geo.set_global_opts(
    title_opts=opts.TitleOpts(title="问题 3 路线图"),
    visualmap_opts=opts.VisualMapOpts(is_show=False)
)

geo.render('问题 3 路线图.html')

# 问题四
# -*- coding: utf-8 -*- # Add this line to declare the file encoding

import pandas as pd
from ortools.linear_solver import pywraplp

top_50_cities = pd.read_csv(r"D:\B\问题 3、4 数据.csv", encoding='GBK')

solver = pywraplp.Solver.CreateSolver('SCIP')

cities = top_50_cities['城市'].tolist()
n = len(cities)
x = {}
for i in range(n):
    x[i] = solver.IntVar(0, 1, f'x[{i}]')

travel_time = top_50_cities.set_index('城市')['交通时间'].to_dict()
travel_cost = top_50_cities.set_index('城市')['交通费用'].to_dict()
sight_time = top_50_cities.set_index('城市')['参观时间'].to_dict()
sight_cost = top_50_cities.set_index('城市')['门票费用'].to_dict()

time_limit = 5760
time_constraints = solver.Constraint(0, time_limit)
for i in range(n):

```

```

        city = cities[i]
        time_constraints.SetCoefficient(x[i], travel_time.get(city, 0) +
sight_time.get(city, 0))

guangzhou_index = cities.index('广州') if '广州' in cities else -1
if guangzhou_index != -1:
    for i in range(n):
        if i != guangzhou_index:
            time_constraints.SetCoefficient(x[i], travel_time.get(cities[i],
0) + sight_time.get(cities[i], 0))

lambda_weight = 0.0002
objective = solver.Objective()
for i in range(n):
    cost = travel_cost.get(cities[i], 0) + sight_cost.get(cities[i], 0)
    objective.SetCoefficient(x[i], 1 - lambda_weight * cost)
objective.SetMaximization()

status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    total_time = 0
    cost = 0
    total_sights = 0
    visited_cities = []

    for i in range(n):
        if x[i].solution_value() == 1:
            city = cities[i]
            total_time += travel_time.get(city, 0) + sight_time.get(city, 0)
            cost += travel_cost.get(city, 0) + sight_cost.get(city, 0)
            total_sights += 1
            visited_cities.append(city)

    print("总游玩时间:", total_time, "分")
    print("总费用:", cost, "元")
    print("总游玩景点数量:", total_sights, "个")
    print("游玩城市列表:", visited_cities)
else:
    print("未找到最优解。")
# 问题四路线图
from pyecharts import options as opts
from pyecharts.charts import Geo
from pyecharts.globals import ChartType, SymbolType

```

```
cities = ['张家口', '烟台', '邢台', '五指山', '无锡', '潍坊', '唐山', '苏州', '石家庄',  
         '沈阳', '上海', '山南', '三亚', '日喀则', '青岛', '秦皇岛', '钦州', '南宁', '那曲',  
         '柳州', '陵水', '林芝', '廊坊', '拉萨', '金华', '济南', '衡水', '桂林', '贵港', '甘孜',  
         '防城港', '崇左', '承德', '成都', '北京', '北海', '保定', '阿里', '阿坝']
```

```
coordinate_override = {  
    '张家口': [114.8875, 40.8244],  
    '烟台': [121.4479, 37.4645],  
    '邢台': [114.5048, 37.0706],  
    '五指山': [109.5169, 18.7751],  
    '无锡': [120.3119, 31.4912],  
    '潍坊': [119.1618, 36.7068],  
    '唐山': [118.1802, 39.6305],  
    '苏州': [120.5832, 31.2983],  
    '石家庄': [114.5149, 38.0428],  
    '沈阳': [123.4315, 41.8057],  
    '上海': [121.4737, 31.2304],  
    '山南': [91.7714, 29.2286],  
    '三亚': [109.5119, 18.2528],  
    '日喀则': [88.8818, 29.2671],  
    '青岛': [121.3119, 35.3512],  
    '秦皇岛': [119.5996, 39.9354],  
    '钦州': [108.6541, 21.9797],  
    '南宁': [108.3669, 22.8170],  
    '那曲': [92.0513, 31.4761],  
    '柳州': [109.4318, 24.3246],  
    '陵水': [110.0375, 18.5050],  
    '林芝': [94.3497, 29.6516],  
    '廊坊': [116.6997, 39.5128],  
    '拉萨': [91.1721, 29.6525],  
    '金华': [119.6474, 29.0791],  
    '济南': [117.1201, 36.6512],  
    '衡水': [115.6702, 37.7389],  
    '桂林': [110.2991, 25.2742],  
    '贵港': [109.5989, 23.1133],  
    '甘孜': [99.9927, 31.6190],  
    '防城港': [108.3490, 21.6930],  
    '崇左': [107.3649, 22.3774],  
    '承德': [117.9634, 40.9515],  
    '成都': [104.0668, 30.5728],  
    '北京': [116.4074, 39.9042],  
    '北海': [109.1201, 21.4811],  
    '保定': [115.4646, 38.8739],
```

```

    '阿里': [80.1055, 32.5011],
    '阿坝': [102.2247, 31.8998],
}

route = ['张家口', '烟台', '邢台', '五指山', '无锡', '潍坊', '唐山', '苏州', '石家庄', '沈阳', '上海',
'山南', '三亚', '日喀则', '青岛', '秦皇岛', '钦州', '南宁', '那曲', '柳州', '陵水', '林芝', '廊坊', '拉
萨', '金华', '济南', '衡水', '桂林', '贵港', '甘孜', '防城港', '崇左', '承德', '成都', '北京', '北海', '
保定', '阿里', '阿坝']

geo = Geo()

geo.add_schema(maptype="china")

for city in cities:
    if city in coordinate_override:
        geo.add_coordinate(city, coordinate_override[city][0], coordinate_override[city][1])
        geo.add("", [(city, 1)], type_=ChartType.EFFECT_SCATTER, symbol_size=10)

geo.add(
    "route",
    [(route[i], route[i+1]) for i in range(len(route)-1)],
    type_=ChartType.LINES,
    effect_opts=opts.EffectOpts(symbol=SymbolType.ARROW, symbol_size=6,
color="red"),
    linestyle_opts=opts.LineStyleOpts(curve=0.2, color="blue", opacity=0.7, width=2),
)

geo.set_series_opts(label_opts=opts.LabelOpts(is_show=False))
geo.set_global_opts(
    title_opts=opts.TitleOpts(title="问题 4 路线图"),
    visualmap_opts=opts.VisualMapOpts(is_show=False)
)

geo.render('问题 4 路线图.html')

# 问题五
import pandas as pd
import numpy as np
from deap import base, creator, tools, algorithms
def load_city_data(file_path):
    city_data = pd.read_csv(file_path, encoding='utf-8')
    return city_data
def calculate_distance(lat1, lon1, lat2, lon2):
    R = 6371

```

```

    phi1 = np.radians(lat1)
    phi2 = np.radians(lat2)
    delta_phi = np.radians(lat2 - lat1)
    delta_lambda = np.radians(lon2 - lon1)
    a = np.sin(delta_phi / 2) ** 2 + np.cos(phi1) * np.cos(phi2) * np.sin(delta_lambda / 2) **
2
    c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
    distance = R * c
    return distance
def calculate_railway_travel_time(distance):
    speed_kmh = 300
    travel_time_hours = distance / speed_kmh
    return travel_time_hours

def fitness_function(individual, city_data, max_time=66):
    total_time = 0
    total_cost = 0
    visited_cities = []
    start_city = city_data.iloc[individual[0]]
    total_time += start_city['建议游玩时间']
    total_cost += start_city['门票']
    visited_cities.append(start_city['名字'])
    for i in range(len(individual) - 1):
        city1 = city_data.iloc[individual[i]]
        city2 = city_data.iloc[individual[i + 1]]
        distance = calculate_distance(city1['Latitude'], city1['Longitude'], city2['Latitude'],
city2['Longitude'])
        travel_time = city1['建议游玩时间'] + calculate_railway_travel_time(distance)
        ticket_cost = city2['门票'] + distance * 0.46 # total_time += travel_time
        total_cost += ticket_cost
        visited_cities.append(city2['名字'])
        if total_time > max_time:
            return total_time, total_cost, len(visited_cities),
    return total_time + calculate_railway_travel_time(
        calculate_distance(city_data.iloc[individual[-1]]['Latitude'],
                           city_data.iloc[individual[-1]]['Longitude'],
                           city_data.iloc[individual[0]]['Latitude'],
                           city_data.iloc[individual[0]]['Longitude'])), total_cost,
    len(visited_cities)

if __name__ == '__main__':
    file_path = r'H:\数学建模\华数杯\解 C 题\转化后附件\地址数据_1.csv' city_data
    = load_city_data(file_path)
    creator.create("FitnessMulti", base.Fitness, weights=(-1.0, 1.0, 1.0))

```



```

creator.create("Individual", list, fitness=creator.FitnessMulti)
toolbox = base.Toolbox()
toolbox.register("indices", np.random.permutation, len(city_data))
toolbox.register("individual", tools.initIterate, creator.Individual, toolbox.indices)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("mate", tools.cxOrdered)
toolbox.register("mutate", tools.mutShuffleIndexes, indpb=0.05)
toolbox.register("select", tools.selNSGA2)
toolbox.register("evaluate", fitness_function, city_data=city_data)
population = toolbox.population(n=100)
algorithms.eaMuPlusLambda(population, toolbox, mu=50, lambda_=100, cxpb=0.7,
mutpb=0.2, ngen=50, verbose=False)
best_individual = tools.selBest(population, k=1)[0]
total_time, total_cost, num_cities_visited = fitness_function(best_individual, city_data)
best_route_cities = [city_data.iloc[idx]['名字'] for idx in
best_individual[:num_cities_visited]]
print("最佳路线: ", best_route_cities)
print("访问的城市数量: ", num_cities_visited)
print("总时间: ", int(total_time))
print("总费用: ", total_cost)

```